

Docker Practical: Data, Volumes, Bind Mounts, Compose & Dev Containers

Docker Concepts

Imagine Docker is a **temporary room**.

You enter the room and keep your things there.



When you destroy the room:

Container
↓
DELETE
↓
Room gone
↓
Things inside gone

That's the **first problem**.

Problem 1: "I don't want my data to disappear"

For example, you create:

student.txt

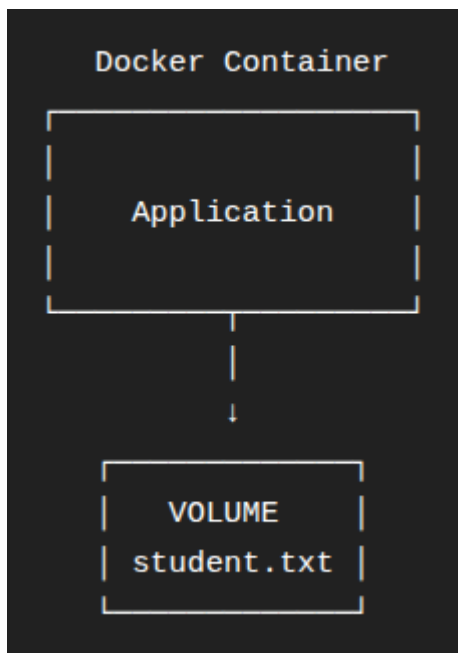
inside the container.

Then you delete the container.

The file disappears.

Solution: Docker Volume

Think of a **volume** as a **storage box outside the room**.



Delete the container:

Container → DELETE ❌

Volume → STILL EXISTS ✅

So we use a **Docker Volume** when we want data to survive container deletion.

Let's Actually Do It

Step 1

Create a volume:

```
sudo docker volume create student-data
```

You just told Docker:

"Create a storage box called `student-data`."

Step 2

Check it:

```
sudo docker volume ls
```

You should see:

Student-data

Delete a specific volume

```
sudo docker volume rm mydata
```

If not possible, delete the container and then delete the volume

Step 3

Create a container and connect the volume:

```
sudo docker run -it --name volume-test -v student-data:/data ubuntu
```

sudo: Runs the command with root/administrator privileges.

docker run: Instructs Docker to create and start a new container.

-it: Runs the container in **interactive mode** with a terminal pseudo-TTY attached. You will land directly at an `root@<container-id>:/#` shell prompt.

- **-i** (interactive): Keeps **STDIN** open so you can type commands.
- **-t** (tty): Allocates a terminal prompt interface.

--name volume-test: Sets the custom name of the container to **volume-test** (instead of Docker generating a random auto-assigned name like `friendly_hellman`).

-v student-data:/data: Mounts a volume named **student-data** into the container at the directory path **/data**.

- If **student-data** doesn't exist on your host machine, Docker automatically creates it.
- Any files created or modified inside **/data** will persist on your host machine even if the container is stopped or deleted.

ubuntu: Specifies the Docker image to use (downloads it from Docker Hub if it isn't available locally).

Step 4

You are now inside the container.

Type:

cd /data

Now you are inside the storage box.

Create a file:

echo "Hello from Docker" > student.txt

Check:

cat student.txt

You should see:

Hello from Docker

Step 5

Exit:

exit

Now delete the container:

sudo docker rm volume-test

The **container is gone**.

But the volume is still there.

Step 6

Create a new container and connect the SAME volume:

sudo docker run -it --name volume-test2 -v student-data:/data ubuntu

Then:

cd /data

and:

ls

You should see:

student.txt

 **That's the entire point of a volume.**

Container gone → Data remains.

Now Forget Volumes for a Moment

The next concept is different.

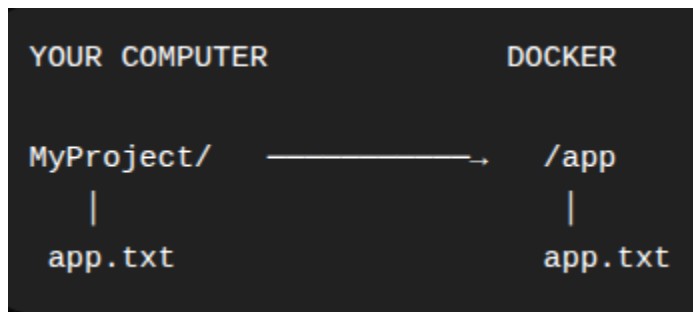
Problem 2: "I have files on my computer. I want Docker to use them."

Suppose your normal Ubuntu computer has:

```
MyProject/  
  app.txt
```

You want Docker to see `app.txt`.

We can connect:



This is called a:

Bind Mount

Practical 2 — Bind Mount

Step 1

Exit your current container:

```
exit
```

Go home:

cd ~

Create a folder:

mkdir MyProject

Enter it:

cd MyProject

Step 2

Create a file on your **normal Ubuntu computer**:

nano app.txt

Write:

**Hello docker bind mount
Docker training**

Save it.

Check:

cat app.txt

You should see your text.

Step 3 — Connect this folder to Docker

Run:

sudo docker run -it -v \$(pwd):/app ubuntu

Don't worry about the whole command.

Look at:

\$(pwd):/app

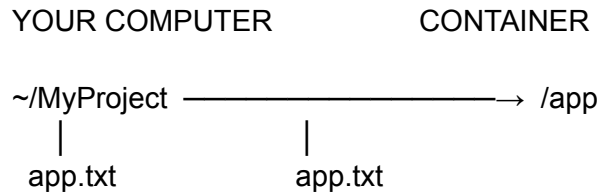
\$(pwd) means:

"My current folder."

You are currently in:

~/MyProject

So Docker creates this connection:



Step 4

Inside the container:

cd /app

Then:

ls

You should see:

app.txt

Then:

cat app.txt

You will see:

Hello docker bind mount

Docker training

 **That's bind mounting.**

Your file is actually on **your computer**, but Docker can access it.

Very Important: Volume vs Bind Mount

This is where you may have been getting confused.

Volume

You say:

"Docker, please create storage for me."

Docker



Volume



Container

Example:

```
-v student-data:/data
```

Bind Mount

You say:

"Docker, use THIS folder from my computer."

My computer



MyProject



Container

Example:

```
-v $(pwd):/app
```

Remember just this:

Volume = Docker's storage

Bind Mount = My computer's folder

Now the Third Concept

You've understood:

1. Container
2. Volume
3. Bind mount

Now imagine you want to run a website.

Normally you might have to type a long command:

`docker run ...`

What if your application needs:

- a container
- a port
- a folder
- a volume
- an environment variable

Typing all that manually becomes annoying.

So Docker gives us:

Docker Compose

Compose allows us to write the configuration in a file.(YML("Yet Another Markup Language."))

Install the file

`sudo apt install docker-compose-plugin`

Check compose version

`sudo docker compose version`

Practical 3 — Docker Compose

Let's make the **simplest possible website**.

Step 1

Go home:

cd ~

Create:

mkdir compose-demo

Enter:

cd compose-demo

Step 2

Create a website folder:

mkdir websites

Now you have:

compose-demo/
└── websites/

Step 3

Create the website:

nano websites/index.html

Put:

```
<!DOCTYPE html>
<html>
<head>
  <title>My Docker Website</title>
</head>
<body>
  <h1>Hello Students!</h1>
  <p>This website is running using Docker.</p>
</body>
</html>
```

Save it.

Step 4 — Create Compose file

Run:

nano docker-compose.yml

Put:

```
services:  
  web:  
    image: nginx  
    ports:  
      - "8080:80"  
    volumes:  
      - ./websites:/usr/share/nginx/html
```

Don't try to memorize this yet.

I'll translate it into English.

This:

services:

means:

"I want to define some services."

This:

web:

means:

"My service is called web."

This:

image: nginx

means:

"Use the Nginx Docker image."

Nginx is going to act as our web server.

This:

ports:

- "8080:80"

means:

Your computer

Container

localhost:8080 → port 80

|
Nginx

So you will access the website through:

localhost:8080

And this:

volumes:

- ./websites:/usr/share/nginx/html

is actually a **bind mount**.

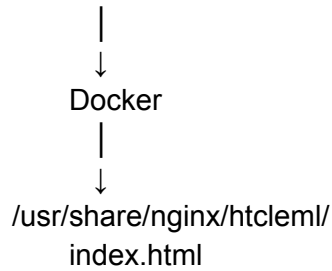
It means:

Your computer

compose-demo/

websites/

index.html



Nginx reads your HTML file.

Step 5 — Start Everything

Run:

`sudo docker compose up`

Docker reads your YAML file.

It sees:

I need Nginx.

I need port 8080 → 80.

I need the websites folder mounted.

Then Docker creates everything.

Step 6 — See Your Website

Open your browser.

Go to:

`http://localhost:8080`

You should see:

Hello Students!

Now you have actually used Docker Compose.

Step 7 — Stop It

Press:

Ctrl + C

Then:

sudo docker compose down

The container is removed.

But your:

websites/index.html

is still on your computer.

Why?

Because the HTML file was on the **host computer** and was bind mounted into Docker.

To bring the website back

Simply run:

cd ~/compose-demo

sudo docker compose up

Then open:

http://localhost:8080

Your website should return.

Last Concept: Dev Container

This one sounds complicated, but it is actually quite simple.

Imagine you are teaching Python.

Normally every student's computer needs:

- Python
- Python libraries
- VS Code extensions
- Correct Python version
- Other tools

One student has Python 3.10.

Another has Python 3.12.

Another is missing a library.



A **Dev Container** solves this by saying:

"Let's create the development environment inside Docker."

VS Code



Docker



Python 3.12



Your development environment

Practical 4 — Dev Container

Step 1

Go home:

cd ~

Create:

mkdir devcontainer-demo

Enter:

cd devcontainer-demo

Step 2

Create a special folder:

mkdir .devcontainer

The structure is:

```
devcontainer-demo/  
└── .devcontainer/
```

Step 3

Create json file(Javascript Object Notation):

nano .devcontainer/devcontainer.json

Put:

```
{  
  "name": "Python Dev",  
  "image": "python:3.12",  
  "customizations": {  
    "vscode": {  
      "extensions": [  
        "ms-python.python"  
      ]  
    }  
  }  
}
```

What does this file say?

This:

```
"name": "Python Dev"
```

gives the development environment a name.

This:

```
"image": "python:3.12"
```

says:

```
    "Use Python 3.12."
```

This:

```
"ms-python.python"
```

says:

```
    "Install the Python extension in VS Code."
```

That's basically it.

Step 4

Open the project:

code .

In VS Code, install the:

Dev Containers extension.

Then:

Ctrl + Shift + P

Search:

Dev Containers: Reopen in Container

Select it.

If any error occurs like,

The current user does not have permission to run `docker`.

Fix it

1. Add your user to the Docker group

Open a normal terminal and run:

`sudo usermod -aG docker $USER`

Enter your Ubuntu password.

2. Log out of Ubuntu

This step is important.

Log out completely from your Ubuntu session and log back in.

You can also restart the computer:

`sudo reboot`

3. Check Docker without `sudo`

After logging back in, open a terminal and run:

`docker ps`

If it works and you don't get:

permission denied

then the problem is fixed.

You can also test:

`docker run hello-world`

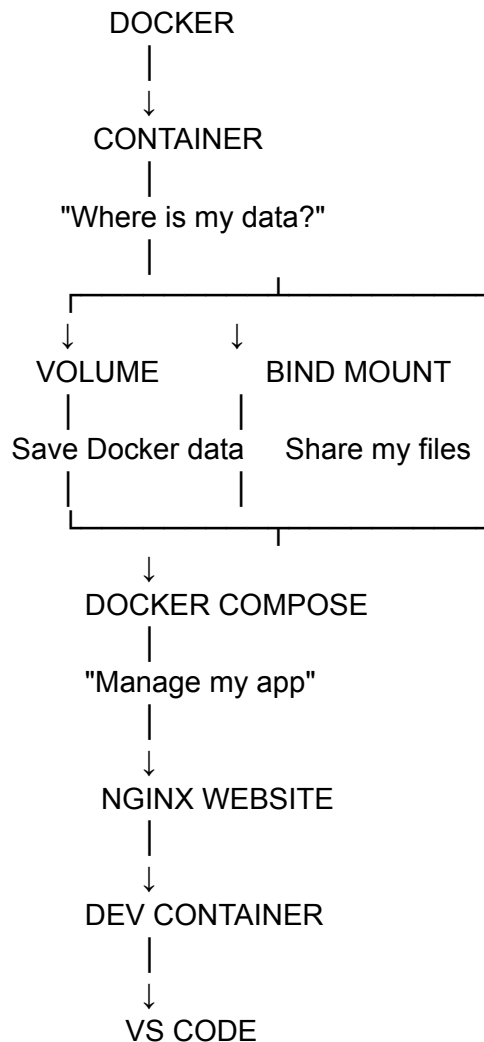
Notice that we are not using `sudo`.

4. Restart VS Code

Completely close VS Code and open it again and do the above tasks again.

0

SUMMARY,



The four sentences students need to remember

1. Volume

"I want Docker-managed storage that survives container deletion."

2. Bind Mount

"I want to connect a folder on my computer to a folder inside Docker."

3. Docker Compose

"I want to define my Docker application in a YAML file and start it easily."

4. Dev Container

"I want Docker to provide my development environment inside VS Code."